

BuySellChain

Sistema d'Asta Trustless su Blockchain Hyperledger Fabric

Documento di Progettazione e Analisi di Sicurezza

Salvucci Franco (Mat. 0368692)
Ionut Georgian Zbirciog (Mat. 0381279)

12 Marzo 2026

Sommario

Il mercato immobiliare tradizionale è caratterizzato da inefficienze e dipendenza da intermediari. BuySellChain propone un sistema decentralizzato basato su **Blockchain** per eliminare l'agente immobiliare fisico. L'architettura su Hyperledger Fabric gestisce tramite Smart Contract la validazione delle offerte e il trasferimento della proprietà digitale. Il sistema garantisce trasparenza assoluta e azzeramento delle commissioni.

Indice

1	Introduzione	2
2	Architettura del Sistema	3
2.1	Gestione dell'Identità e Sicurezza	4
2.2	User Requirements Definition	5
2.3	Frontend	7
2.4	Backend (Security Gateway)	8
2.4.1	Authentication & Authorization	8
2.4.2	Modello Dati & Interfaccia RESTful API	9
2.4.3	Logica di Bussiness	16
2.5	Blockchain Layer	18
2.6	Database Off-Chain	19
3	Definizione dell'infrastruttura e delle Zone di Sicurezza	20
3.1	Zona DMZ (Demilitarized Zone)	20
3.2	Zona IT / Enterprise (Restricted Zone)	21
3.3	Asset Inventory e Analisi delle Criticità	22
3.3.1	Endpoint del Security Gateway	22
3.4	Superficie Osservabile, Inferenza e Vettori di Attacco	23
3.5	Grafo dell'Infrastruttura e Flussi di Comunicazione	24
3.6	Dall'Architettura alla Baseline Comportamentale (Operating Envelope) . .	26
3.6.1	Definizione della Baseline Qualitativa	26
4	Logger	27
4.1	Definizione Sistema di Logging	27
4.2	Esempi di Log e Analisi Comportamentale	29
4.2.1	User Agent Metadata Role	30
4.3	Monitoraggio Autenticazione & Admin	30
	Bibliografia	32

Capitolo 1

Introduzione

Il sistema BuySellChain mira a decentralizzare il mercato immobiliare tramite un servizio di aste online. Come verrà illustrato, l'agente si limita alla registrazione dell'asta e non svolge più il ruolo di intermediario tipico del modello tradizionale. In questa architettura, l'intermediazione è affidata alla Blockchain, incaricata di validare ogni offerta. Questo approccio garantisce trasparenza, sicurezza e una riduzione dei costi operativi, eliminando la necessità di intermediari tradizionali e consentendo agli utenti di partecipare direttamente al processo di compravendita.

Il sistema da noi proposto ha come obiettivo quello di implementare un sistema di aste **a busta chiusa**. In questo modello di aste, ciascun competitor non è a conoscenza delle offerte attuali degli altri competitor, garantendo la **segretezza** delle offerte, ciò comporta maggiore competizione tra gli offerenti.

La documentazione di questo progetto è organizzata come segue:

Architettura del Sistema Saranno presentate l'architettura e le tecnologie utilizzate, con particolare attenzione al backend e alla Blockchain.

Analisi del Perimetro di Attacco Verranno delineati il perimetro di attacco e le principali minacce.

Possibili Attacchi e Come Difendersi Infine, saranno discussi i possibili attacchi e le contromisure adottate per mitigarli.

Capitolo 2

Architettura del Sistema

L'architettura di **BuySellChain** adotta un approccio ibrido che integra la flessibilità del Web 2.0 con la sicurezza del Web 3.0. Il sistema è progettato per garantire la disintermediazione della compravendita immobiliare, come richiesto dagli obiettivi di progetto.

Frontend: Interfaccia utente realizzata con **Jinja2** e **Alpine.js**, che fornisce reattività lato client senza richiedere una build pipeline complessa. La responsabilità è limitata alla presentazione dei dati e all'interazione utente, senza contenere logica di business critica.

Backend (Security Gateway): Server applicativo implementato in **Python** con il microframework **Flask**, che funge da middleware di sicurezza tra il client pubblico e la rete privata Blockchain. Questa componente ha il ruolo critico di sanitizzare gli input, gestire le sessioni, implementare il controllo degli accessi basato su ruoli (RBAC) e prevenire l'esposizione diretta delle API del Ledger. Si occupa dell'autenticazione degli utenti tramite credenziali verificate sul database off-chain e della generazione di JWT per l'autorizzazione stateless delle operazioni successive.

Blockchain Layer: Il nucleo del sistema, basato su **Hyperledger Fabric** [1], che funziona come database distribuito e immutabile per le operazioni critiche di compravendita. Questo livello è accessibile esclusivamente tramite il client Lisp fornito dal docente, che incapsula la logica di interazione con il chaincode e la gestione delle identità crittografiche (MSP). Il meccanismo di consenso distribuito valida le transazioni secondo le regole definite negli Smart Contract, garantendo l'integrità e la non-ripudiabilità delle operazioni.

Database Off-Chain: Un database relazionale **PostgreSQL** utilizzato per la persistenza dei dati sensibili e non critici per il consenso distribuito, come i profili utente, le credenziali di accesso al portale web e l'anagrafe del sistema. L'accesso è mediato attraverso **SQLAlchemy**, un'astrazione ORM che garantisce protezione contro attacchi SQL injection. Questa separazione garantisce la conformità alle normative sulla privacy (GDPR), evitando la scrittura di dati sensibili sul Ledger immutabile. Le password sono protette mediante l'algoritmo di hashing bcrypt con salt unico per ogni utente.

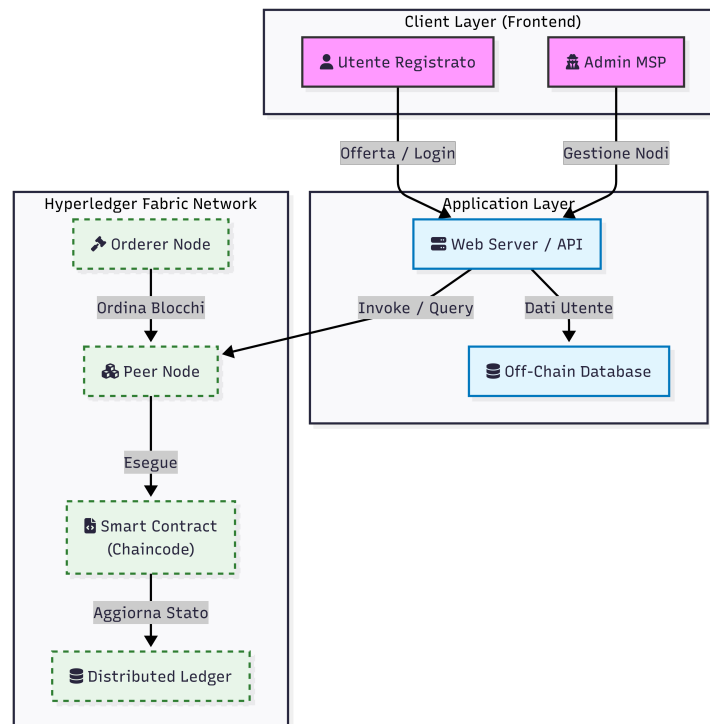


Figura 2.1: Schema ad alto livello dell'architettura ibrida Web 2.0 / Web 3.0

2.1 Gestione dell'Identità e Sicurezza

Poiché Hyperledger Fabric è una rete di tipo *permissioned*, a differenza delle blockchain pubbliche, tutti i partecipanti devono essere identificati e autorizzati prima di poter interagire con il ledger. In questo contesto, la fiducia non è anonima ma basata su identità crittografiche verificate.

Per gestire questo livello di autorizzazione, l'architettura si basa su un componente fondamentale:

Definizione (*Membership Service Provider (MSP)*). *L'MSP è il componente astratto del sistema che definisce le regole di validazione delle identità. Esso agisce come un'autorità di certificazione logica che consente alla rete di riconoscere e considerare affidabili i certificati digitali presentati dai peer e dagli utenti.*

Il funzionamento operativo coinvolge la cooperazione tra MSP e le *Certificate Authorities (CA)*:

1. **Rilascio dell'Identità:** Le CA generano una coppia di chiavi (pubblica e privata) e rilasciano un certificato X.509 che rappresenta l'identità digitale dell'attore.
2. **Verifica (Ruolo dell'MSP):** Quando un attore (es. un Peer) firma digitalmente una transazione usando la propria chiave privata, l'MSP interviene per verificare che tale certificato sia stato emesso da una CA fidata e che l'attore possieda i permessi necessari per quella specifica operazione (es. avallare una transazione).

In questo scenario, il client Lisp incapsula tale processo; di conseguenza l'attenzione è rivolta all'autenticazione dell'utente presso il server Web (Security Gateway).

2.2 User Requirements Definition

In questa sezione vengono definiti gli attori che interagiscono con la piattaforma **BuySell-Chain** e le azioni a loro consentite. Il sistema è progettato per operare in un ambiente *trustless* basato su Hyperledger Fabric, eliminando la necessità di intermediari umani nella fase di negoziazione.

User Classes and Characteristics

Amministratore di Sistema: Gestisce la piattaforma, monitora anomalie e alert di sicurezza. Dispone di una dashboard dedicata per osservabilità, auditing e interventi correttivi.

Utente Registrato: Attore abilitato alla compravendita; lo stesso account può ricoprire ruoli distinti:

- **Venditore (Proprietario dell'asset):** Possiede l'immobile digitalizzato sul registro e avvia l'asta definendone i parametri.
- **Offerente (Bidder):** Partecipa all'asta inviando offerte firmate e vincolanti secondo le regole dello Smart Contract.

Utente Non Autenticato: Visitatore esterno. Senza account può consultare le aste e gli immobili pubblicati, senza possibilità di interazione avanzata.

User Actions and Use Cases

Le azioni riflettono la logica di business gestita dal server Web e dagli Smart Contract.

- **Azioni Utente Non Autenticato:**
 - Consultare la landing page del progetto.
 - Registrarsi o accedere.
 - Effettuare un login sicuro.
- **Azioni dell'Amministratore (Monitoraggio e Sicurezza):**
 - Monitorare gli alert su tentativi di violazione dell'integrità (mismatch crittografici).
 - Analizzare i log delle offerte rifiutate per violazioni delle finestre temporali (anti-frontrunning).
- **Azioni dell'Utente Registrato — Venditore:**
 - Avviare un'asta per un immobile già digitalizzato.
 - Impostare la finestra temporale dell'asta (inizio/fine) gestita dallo Smart Contract.

- Visualizzare in tempo reale sul Ledger lo stato delle offerte ricevute.
- Ricevere la notifica di aggiudicazione e il trasferimento della proprietà digitale.
- **Azioni dell'Utente Registrato — Offerente:**
 - Consultare le aste immobiliari attive.
 - Inviare un'offerta firmata; lo Smart Contract verifica l'incremento minimo richiesto.
 - Verificare la propria posizione in graduatoria (anonimizzata o pubblica secondo la logica di business).
 - Visualizzare l'esito della validazione (accettata/rifiutata per motivi temporali o di importo).

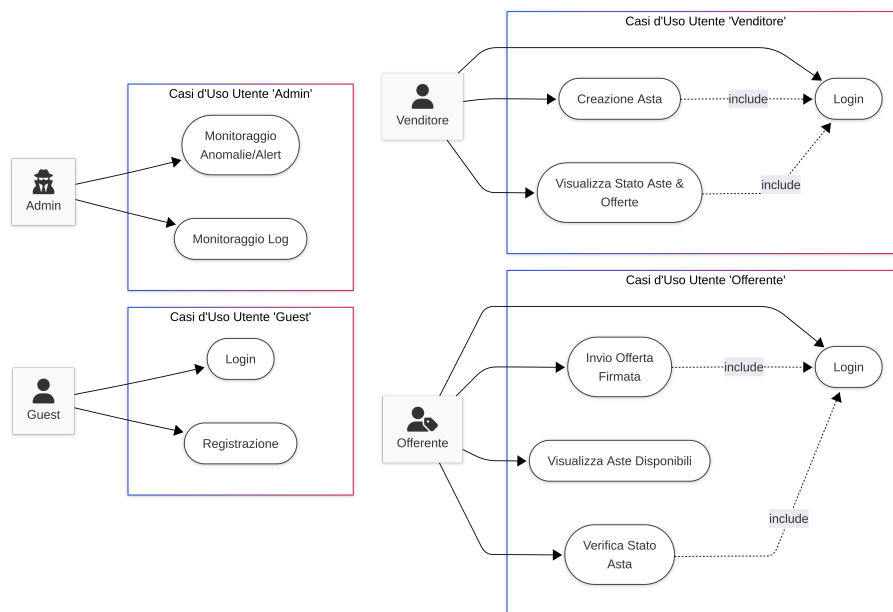


Figura 2.2: Azioni degli utenti all'interno della piattaforma

2.3 Frontend

Per la realizzazione del frontend è stato adottato il motore di templating **Jinja2** [2], integrato nativamente in Flask, per la generazione dinamica delle pagine HTML lato server. Jinja2 consente di creare template modulari e riutilizzabili, separando la logica di presentazione dai dati applicativi e facilitando la manutenzione del codice.

Per l'interattività lato client, è stato scelto il framework leggero **Alpine.js** [3], che fornisce reattività senza richiedere una build pipeline complessa. Alpine.js permette di dichiarare il comportamento dinamico direttamente nel markup HTML mediante direttive semplificate, risultando ideale per applicazioni web moderne ma senza la complessità di framework come React o Vue.

Questa combinazione garantisce un frontend performante, mantenibile e facilmente integrabile con il backend Flask, preservando la semplicità dell'architettura complessiva.

Il frontend consiste delle seguenti route:

/login Pagina di autenticazione per l'accesso degli utenti registrati.

/signin Pagina di registrazione per i nuovi utenti.

/ Pagina home nella quale vengono presentate tutte le aste disponibili in formato grid. Ogni card della griglia mostra la foto principale dell'immobile, il titolo dell'asta, il prezzo di partenza e lo stato. Gli utenti non autenticati possono consultare le aste ma non partecipare.

/faq Pagina statica con le domande frequenti e le informazioni di contatto per l'assistenza.

/auctions/create Pagina per la creazione di una nuova asta. Accessibile solo dai venditori che abbiano precedentemente registrato almeno un immobile. Consente di impostare i parametri temporali (data inizio e fine), il prezzo di partenza e l'incremento minimo.

/auctions/{id} Route dinamica generata per ciascun'asta. Visualizza i dettagli completi dell'immobile, lo stato dell'asta in tempo reale, la graduatoria delle offerte (ove applicabile) e consente agli utenti autenticati di partecipare inviando un'offerta.

/profile Pagina del profilo utente nella quale sono disponibili i dati personali, la cronologia delle aste create (per i venditori) e la cronologia delle offerte inviate (per gli offerenti).

2.4 Backend (Security Gateway)

In questa sezione verrà analizzato il backend insieme alla logica di business. Questa sezione descrive nel dettaglio il componente architetturale che funge da ponte sicuro tra l'interfaccia pubblica (Frontend) e la rete privata (Blockchain Layer). Il Security Gateway non si limita a inoltrare le richieste, ma implementa la logica applicativa necessaria per garantire l'integrità del sistema, la gestione delle identità e la validazione preliminare delle transazioni, proteggendo il Ledger da accessi non autorizzati o malformati.

La tecnologia scelta per l'implementazione del backend è **Python** [4], utilizzando il microframework **Flask** [5, 6]. Questa scelta è motivata dalla necessità di avere un ambiente di sviluppo rapido, flessibile e con un ecosistema ricco di librerie per la crittografia e la gestione delle reti. Il server Flask espone una serie di API RESTful che vengono consumate dal client Frontend.

2.4.1 Authentication & Authorization

La sicurezza di **BuySellChain** si articola su due livelli distinti ma interconnessi: l'autenticazione degli utenti e la gestione dei permessi operativi.

- **Autenticazione (Identity Verification):** Il Security Gateway agisce come punto d'ingresso unico. Verifica le credenziali dell'utente (username e password) consultando il database off-chain. Una volta validata l'identità, il sistema mappa l'utente Web 2.0 a un'identità digitale riconosciuta dalla rete Fabric, garantendo che ogni operazione sul ledger sia tracciabile e attribuibile a un attore specifico.
- **Autorizzazione (Access Control):** Dopo l'autenticazione, il sistema applica politiche di controllo degli accessi basate sui ruoli (RBAC). Il backend verifica se l'utente possiede i privilegi necessari per invocare specifiche transazioni (es. solo un utente con ruolo "Venditore" può inizializzare un'asta).

Il Security Gateway funge da *Policy Enforcement Point*, intercettando le richieste e verificando che il richiedente possieda i privilegi necessari (es. solo il proprietario di un asset può metterlo all'asta) prima di inoltrare qualsiasi comando alla Blockchain.

Una volta completata con successo la fase di login, il server genera e firma un *JSON Web Token* (JWT) [7]. Questo token funge da passaporto digitale per l'utente e contiene, nel suo *payload*, informazioni essenziali quali l'identificativo utente, i ruoli assegnati e la data di scadenza della sessione.

Il client (Frontend) riceve il token e lo memorizza in modo sicuro. Per ogni successiva interazione con le API del sistema, il client deve includere il JWT nell'header della richiesta HTTP (solitamente nell'header **Authorization: Bearer <token>**). Questo meccanismo *stateless* permette al backend di validare l'identità del richiedente e verificare le autorizzazioni per ogni singola operazione senza dover interrogare continuamente il database delle credenziali.

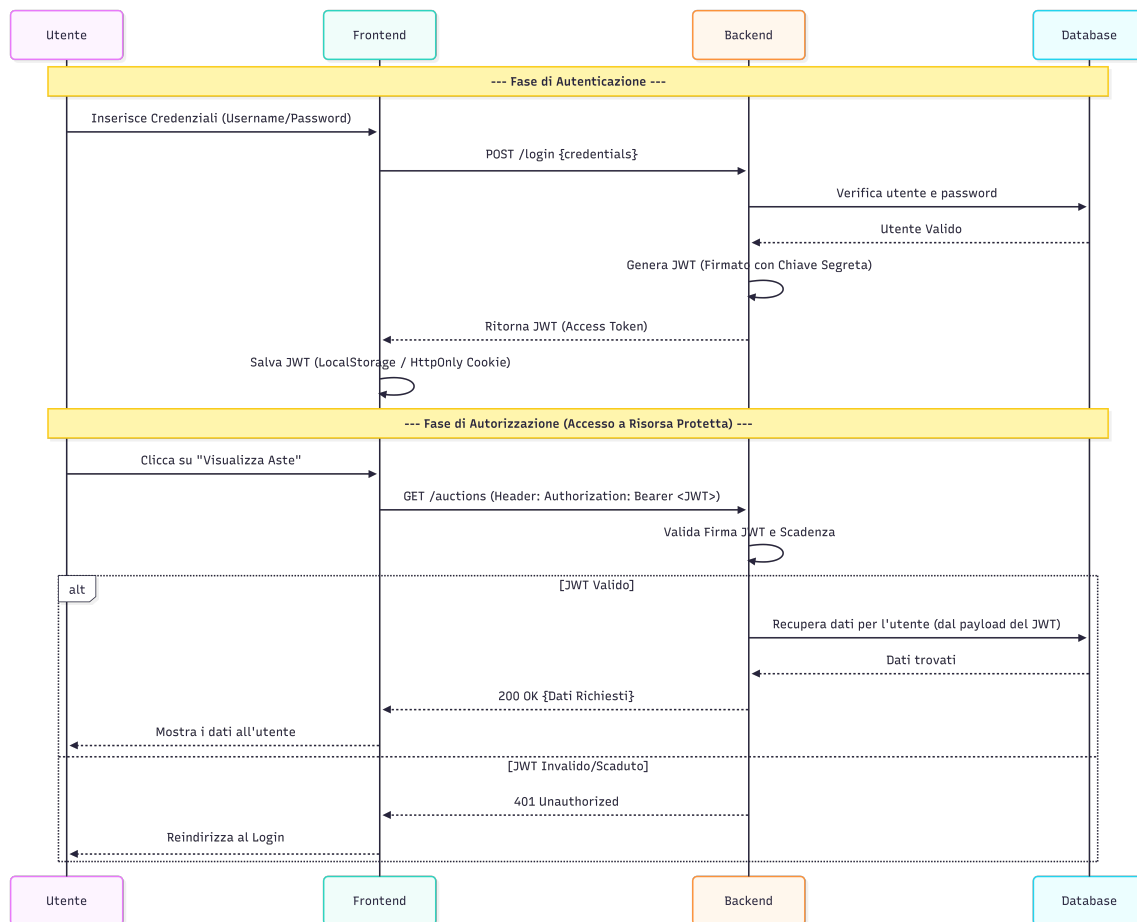


Figura 2.3: Flusso di autenticazione/autorizzazione

2.4.2 Modello Dati & Interfaccia RESTful API

Poiché Hyperledger Fabric mantiene il world state come mappa di coppie (**key**, **value**), a livello applicativo è necessario definire: convenzioni di naming per le chiavi (namespace e identificatori), classi/entità applicative e un formato di serializzazione deterministico (es. JSON).

Le API di base fornite dal prof. Arcieri sono pubblicate come endpoint REST. Il Security Gateway riceve le richieste HTTP della piattaforma web e le traduce in comandi per il client Lisp verso la rete Fabric. Ad esempio, una chiamata HTTP `POST /api/v1/make_bid` viene mappata al comando `AddKV`. Il client invoca quindi le funzioni di *chaincode* per creare, leggere, aggiornare o eliminare coppie (**key**, **value**) sul ledger.

Le primitive disponibili sono:

AddKV: Inserisce una nuova coppia (**key**, **value**) in una classe applicativa.

GetKV: Restituisce il **value** associato a una **key** di una classe.

GetKeyHistory: Restituisce la cronologia immutabile delle operazioni su una **key**.

DelKV: Elimina la coppia (**key**, **value**) associata a una **key**.

GetClasses: Restituisce l'elenco delle classi disponibili.

GetNumKeys: Restituisce il numero di coppie presenti in una classe.

GetKeys: Restituisce l'elenco delle chiavi presenti in una classe.

Listing 2.1: Envelope di richiesta alle API base

```
1 {  
2   "cmd": "AddKV | GetKV | GetKeyHistory | DelKV | GetClasses |  
        GetNumKeys | GetKeys",  
3   "class": "NomeClasse",  
4   "key": "Key",  
5   "value": { ... }  
6 }
```

Definizione Modello Dati

Di seguito vengono presentate le definizioni strutturali delle classi dati che costituiscono il modello di persistenza distribuito. Ogni tabella rappresenta un'entità logica memorizzata in modo immutabile sulla Blockchain, specificando i campi, i tipi di dato e le relazioni chiare necessarie per garantire l'integrità referenziale e la tracciabilità delle operazioni di compravendita immobiliare all'interno del Ledger di Hyperledger Fabric.

Campo	Tipo	Note
id	string	Identificativo univoco
ownerId	string	Riferimento all'utente proprietario
title	string	Titolo descrittivo immobile
location	string	Indirizzo fisico
description	string	Descrizione immobile
currentAuctionId	string null	ID dell'asta attiva (se presente)
createdAt	int (epoch)	Data creazione
status	enum	ACTIVE, LOCKED, TRANSFERRED

Tabella 2.1: Definizione della Classe Asset

Campo	Tipo	Note
id	string	Identificativo univoco offerta
auctionId	string	Riferimento all'asta
bidderId	string	Riferimento all'offerente
amount	number	Importo offerto
timestamp	int (epoch)	Data e ora offerta
status	enum	APPROVED, REJECTED
rejectReason	string null	Motivo del rifiuto (se applicabile)

Tabella 2.2: Definizione della Classe Bid

Campo	Tipo	Note
id	string	Identificativo univoco dell'asta
assetId	string	Riferimento all'asset in vendita
sellerId	string	Riferimento al venditore
startTime	int (epoch)	Data inizio asta
endTime	int (epoch)	Data fine asta
startPrice	number	Prezzo di partenza
minIncrement	number	Incremento minimo rilancio
highestBidId	string null	ID dell'offerta vincente attuale
highestBidAmount	number null	Importo dell'offerta vincente
bidsCount	int	Contatore totale offerte
status	enum	ACTIVE, CLOSED, CANCELLED, LOCKED

Tabella 2.3: Definizione della Classe Auction

Pertanto, ipotizzando un'analogia con le *tabelle SQL* per fini rappresentativi, la struttura delle classi può essere visualizzata come nello schema seguente:

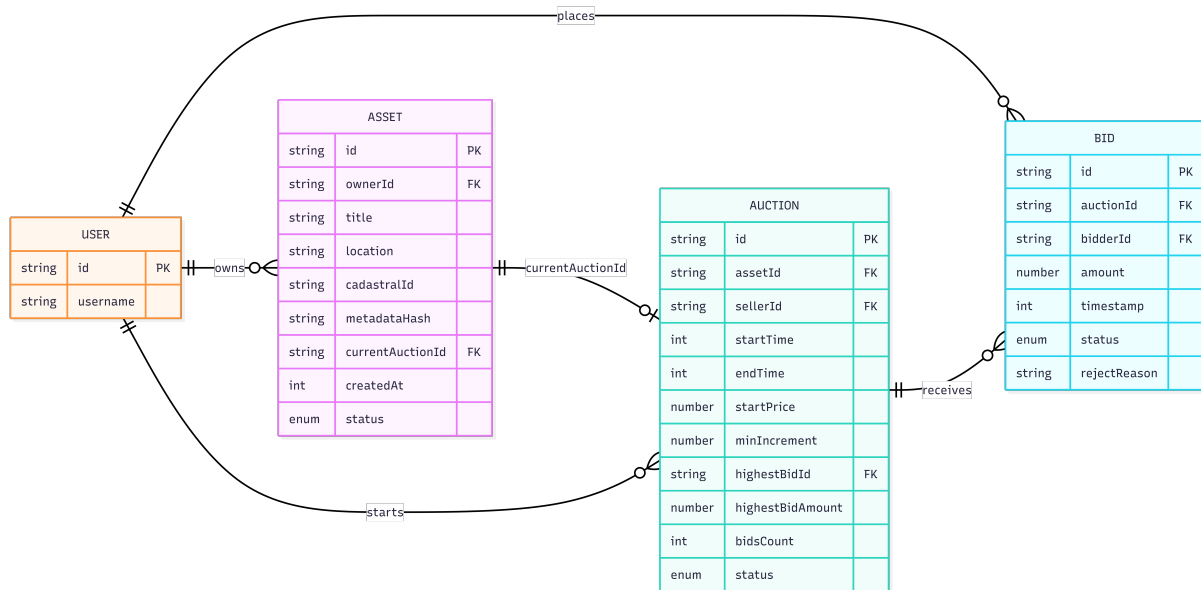


Figura 2.4: Schema logico delle tabelle su Fabric

La figura illustra le relazioni logiche tra le entità. L'utente (User) è l'attore principale: può possedere molteplici Asset, avviare diverse Auction e partecipare attivamente inviando Bid. Gli Asset possono essere associati a una Auction attiva (relazione 1:1 opzionale), mentre ogni Auction è legata univocamente a un bene specifico e funge da contenitore per tutte le offerte ricevute.

Questo modello relazionale, pur essendo implementato su un archivio chiave-valore non relazionale, è progettato per garantire la completa tracciabilità della filiera di vendita (provenance). Mantenendo riferimenti incrociati tra venditori, offerenti, beni e stati

delle aste, il sistema permette di ricostruire in modo deterministico e immutabile l'intera cronologia di una transazione immobiliare, dall'inserimento dell'asset fino all'aggiudicazione finale, assicurando così la massima trasparenza e integrità dei dati.

Definizione Interfaccia RESTful API

Definita la struttura logica dei dati, il passo successivo consiste nell'esposizione dell'interfaccia RESTful API. Nella fase attuale di sviluppo (sperimentale), il server Web è configurato per l'ascolto sulla porta locale standard 5000. L'endpoint base per tutte le richieste API è:

`http://localhost:5000/api/v1/`

Per ridurre la complessità infrastrutturale in ambiente di test, la comunicazione avviene attualmente su protocollo HTTP non cifrato.

I principali endpoint sono:

POST /auth/signin Registrazione di un nuovo utente. La richiesta crea un'identità sul database off-chain.

POST /auth/login Autenticazione utente. Verifica le credenziali e restituisce il token JWT necessario per autorizzare le chiamate successive.

POST /assets Registrazione di un nuovo immobile sul ledger. Richiede che l'utente sia autenticato e abbia il ruolo di "Venditore". Internamente invoca **AddKV** per la classe **Asset**.

GET /assets Recupero dettagli di tutti gli assets. Internamente invoca **GetKeys** e **GetKV** per la classe omonima.

GET /assets/{id} Recupero dettagli di uno specifico asset. Internamente invoca **GetKV** per la classe omonima.

GET /assets/user Recupero dettagli di tutti gli asset posseduti dall'utente autenticato. Internamente invoca **GetKeys** e **GetKV** per la classe **Asset**, filtrando i risultati in base all'ID del proprietario recuperato tramite **JWT**.

POST /auctions Creazione di una nuova asta per un asset posseduto dall'utente. Il sistema verifica la proprietà dell'asset e imposta lo stato iniziale dell'asta. Invoca **AddKV** per la classe **Auction**.

GET /auctions Elenco di tutte le aste presenti a sistema. Internamente utilizza **GetKeys**

GET /auctions/status/{status} Filtraggio delle aste per stato (es. **ACTIVE**, **CLOSED**). Internamente utilizza **GetKeys** e **GetKV** per la classe **Auction**.

GET /auctions/{id} Recupero dettagli di una specifica asta. Internamente invoca **GetKV** per la classe omonima.

GET /auctions/bids/{id} Recupero di tutte le offerte di una specifica asta. Internamente invoca **GetKeys** e **GetKV** per la classe **Bids**, filtrando i risultati in base all'ID dell'asta.

DELETE /auctions/{id} Cancellazione di una specifica asta. Internamente invoca **DelKV** per la classe omonima.

- POST /bids** Invio di un'offerta per un'asta attiva. La logica di backend verifica che l'asta sia aperta, che l'offerta superi l'importo corrente più l'incremento minimo, e che l'offerente non sia il venditore stesso. Se valida, registra l'offerta tramite **AddKV** sulla classe **Bid** e aggiorna lo stato dell'asta.
- GET /bids/user** Recupera tutte le offerte inviate dall'utente autenticato. Internamente invoca **GetKeys** e **GetKV** per la classe **Bid**, filtrando i risultati in base all'ID dell'offerente recuperato tramite **JWT**.
- GET /bids/latest/{id}** Recupero dell'ultima offerta valida fatta dall'utente per una specifica asta. Internamente invoca **GetKeys** e **GetKV** per la classe **Bid**, filtrando i risultati in base all'ID dell'asta.
- GET /admin/users** Restituisce l'elenco completo degli utenti registrati nel database off-chain e il loro stato (attivo/sospeso), per permettere all'amministratore di gestire le utenze.
- GET /admin/stats** Fornisce metriche aggregate del sistema, come il numero totale di aste concluse, il volume delle transazioni e il numero di asset registrati, invocando **GetNumKeys** sulle diverse classi.

Per mantenere una vista compatta delle route principali e delle ultime estensioni implementate, si riporta anche una sintesi tabellare.

Metodo	Endpoint	Note operative
POST	/api/v1/auth/signin	Registrazione utente (bidder/seller), validazione email e CF seller
POST	/api/v1/auth/login	Login e rilascio JWT
GET/PUT	/api/v1/auth/profile	Lettura/aggiornamento profilo autenticato
POST	/api/v1/assets	Creazione asset con upload immagine
GET	/api/v1/assets/user	Asset posseduti dall'utente autenticato
POST	/api/v1/auctions	Creazione asta con lock dell'asset
POST	/api/v1/auctions/lock/{auction_id}	Transizione stato a LOCKED
POST	/api/v1/auctions/close/{auction_id}	Chiusura asta e determinazione vincitore
GET	/api/v1/auctions/getAuctionHistory/{auction_id}	Storico offerte filtrato per ruolo/stato asta
POST	/api/v1/bids	Inserimento offerta con validazioni importo/-tempo/stato
GET	/api/v1/bids/total/{auction_id}	Totali offerte: complessive, valide e rifiutate
GET	/api/v1/admin/stats	Metriche aggregate dashboard admin

Tabella 2.4: Sintesi API REST principali (aggiornata alle ultime funzionalita)

Listing 2.2: Esempio di richiesta AddKV per aggiungere un Asset

```
1  "cmd": "AddKV",
2  "class": "Assets",
3  "key": [
4      "ef25bc38bb1182a80bd7d4b01bc45d9a34/"
5      "b84c939bf944459dc44a79c0bf6259"
6  ],
7  "value": {
8      "asset_type": "villa",
9      "created_at": "2026-03-11T11:59:41.544210",
10     "current_auction_id": null,
11     "description": "Villetta indipendente",
12     "id": "ef25bc38bb1182a80bd7d4b01bc45d/"
13     "9a34b84c939bf944459dc44a79c0bf6259",
14     "location": "Roma",
15     "owner_id": "32503db9ad32a1b8999988977b40d/"
16     "fad523ec82a8a1251ce8c05c85842471b0c",
17     "price": 50000,
18     "size": 120,
19     "status": "active",
20     "title": "Villa"
21 }
```


Listing 2.3: Esempio di richiesta AddKV per creare un'asta

```
1 {
2   "cmd": "AddKV",
3   "class": "Auctions",
4   "key": "1781938b47129ab1e6059a4ec5cca/
5 f5336e69406de0852683e765c799cee7d04",
6   "value": {
7     "id": "1781938b47129ab1e6059a4ec5cca/
8 f5336e69406de0852683e765c799cee7d04",
9     "assetId": "ef25bc38bb1182a80bd7d4b01bc45d/
10 9a34b84c939bf944459dc44a79c0bf6259",
11     "asset_title": "Villa",
12     "asset_description": "Villetta indipendente",
13     "sellerId": "32503db9ad32a1b8999988977b40dfad/
14 523ec82a8a1251ce8c05c85842471b0c",
15     "bid_count": 0,
16     "high_bid_amount": null,
17     "high_bid_id": null,
18     "image_url": "/static/uploads/assets/seller_id/asset_id/primary
19 -8b97d3c3f46f4546ac9c8407496901cb.png",
20     "min_incr": 15000,
21     "start_time": "2026-03-11T15:00:00",
22     "end_time": "2026-03-11T16:00:00",
23     "starting_price": 25000,
24     "status": "ACTIVE"
25 }
```

Listing 2.4: Esempio di richiesta AddKV per aggiungere un'offerta (Bid)

```
1 {
2   "cmd": "AddKV",
3   "class": "Bids",
4   "key": "8f9b21b63558e4154fd8f0bf8cbaf7b/
5 0aaa7170dbd7e0565e0af60ffc782ecd4",
6   "value": {
7     "id": "8f9b21b63558e4154fd8f0bf8cbaf7b/
8 0aaa7170dbd7e0565e0af60ffc782ecd4",
9     "auctionId": "1781938b47129ab1e6059a4ec5cca/
10 f5336e69406de0852683e765c799cee7d04",
11     "bidderId": "01b1de7136182f016755e0bac1a61f60/
12 92df799e9fea2bf9848e1c682a73c9cf",
13     "bid_amount": 80000,
14     "timestamp": 2026-03-11T12:11:00.764654,
15     "status": "approved",
16     "reason": null
17   }
18 }
```

2.4.3 Logica di Bussiness

La piattaforma implementa un modello di gestione degli attori e delle aste articolato in fasi distinte:

Registrazione e Verifica dei Venditori Un utente che desideri partecipare alle aste come venditore deve completare una procedura di registrazione qualificata. Oltre alle informazioni anagrafiche standard (nome, cognome, email, numero di telefono), il sistema richiede l'inserimento del codice fiscale, necessario per garantire l'identificazione univoca e conforme alle normative sulla tracciabilità delle operazioni immobiliari. Una volta sottomessa la richiesta, un messaggio di notifica viene inviato agli amministratori del sistema, i quali hanno la responsabilità di validare l'identità del richiedente e approvare o rifiutare l'iscrizione al ruolo di venditore.

Struttura Temporale delle Aste Ogni asta è caratterizzata da un intervallo temporale, espresso in giorni, definito dal venditore in fase di creazione dell'asta. Durante questo intervallo temporale, gli offerenti possono partecipare ed inviare offerte. Tale intervallo è suddiviso logicamente in finestre temporali di durata predeterminata, al fine di implementare meccanismi anti-frontrunning e garantire equità nella partecipazione.

Vincoli sulle Offerte Per ogni finestra temporale, ciascun offerente può inviare un numero predefinito di offerte, preimpostato da sistema. Questa limitazione impedisce comportamenti speculativi e riduce la complessità computazionale del sistema. Le offerte rimangono nascoste agli altri partecipanti fino al termine della relativa finestra temporale, garantendo così l'anonimato del processo di bidding.

Aggiornamento dello Stato dell'Asta Al termine di ogni finestra temporale, il sistema esegue automaticamente un'operazione di consolidamento dello stato dell'asta:

- Lo stato dell'asta transisce in **LOCKED**, impedendo temporaneamente l'invio di nuove offerte.
- Vengono valutate tutte le offerte ricevute durante la finestra.
- I campi `highestBidAmount`, `bidsCount` e `highBidId` vengono aggiornati per riflettere l'offerta vincente al momento e il conteggio totale delle offerte effettuate.
- Il venditore acquisisce la possibilità di modificare il parametro `minIncrement`, adattando dinamicamente il valore richiesto per i successivi rilanci.

Una volta completato l'aggiornamento, l'asta ritorna allo stato **ACTIVE**, rendendo di nuovo disponibile la partecipazione per la finestra temporale successiva.

La finestra temporale dell'asta viene definita dagli admin, ed è preimpostata a 12 ore (dalle 8.00 AM fino alle 20.00 PM).

Questo meccanismo garantisce una distribuzione equa degli intervalli di partecipazione e facilita l'implementazione di politiche anti-frontrunning, in quanto ogni finestra temporale costituisce un perimetro di validazione indipendente per le offerte ricevute.

Storico Offerte con Visibilita per Ruolo Tra le ultime modifiche introdotte e stata aggiunta la route `/api/v1/auctions/getAuctionHistory/{auction_id}`, che espone la cronologia delle offerte usando `GetKeyHistory` e applica filtri lato backend:

- il venditore visualizza tutte le offerte (incluse quelle **REJECTED**);
- gli altri utenti, durante lo stato **ACTIVE**, visualizzano solo le proprie offerte;
- quando l'asta transisce in **LOCKED/CLOSED**, gli utenti non venditori vedono anche le offerte altrui (solamente quelle validate, non le rifiutate.)

Chiusura dell'Asta e calcolo del Vincitore Alla chiusura dell'intervallo temporale definito per l'Asta, il sistema esegue le seguenti operazioni:

- Lo stato dell'asta transisce in **CLOSED**, chiudendo definitivamente l'invio di nuove offerte
- Viene calcolato il vincitore dell'asta (calcolato come offerta più alta registrata a sistema)
- **In caso di contenzioso** ogni partecipante potrà vedere, sulla pagina principale dell'asta, lo storico di **tutte** le offerte inserite a sistema con relativo **timestamp**, che va ad indicare esattamente quando l'offerta è stata registrata sulla Blockchain

Notifiche Email di Esito Asta Alla chiusura dell'asta (route `/api/v1/auctions/close/{auction_id}`), il backend invia notifiche email:

- email al vincitore con importo finale e contatto del venditore;
- email al venditore con dettaglio aggiudicazione;
- email ai partecipanti non vincitori con motivazione sintetica di mancata aggiudicazione.

2.5 Blockchain Layer

L'ultimo layer dell'architettura è rappresentato dalla blockchain Hyperledger Fabric, che svolge due ruoli fondamentali nel sistema: fungere da database distribuito e immutabile, e validare le transazioni effettuate dagli utenti secondo le regole definite negli Smart Contract.

In questo contesto, Fabric agisce come sistema di registrazione ufficiale (ledger) per tutte le operazioni critiche di compravendita: dalla registrazione degli asset immobiliari, alla creazione delle aste, fino alla registrazione delle offerte. La natura distribuita e permissioned della rete garantisce che ogni transazione sia validata da più peer secondo il meccanismo di consenso configurato, assicurando così l'integrità e la non-ripudiabilità delle operazioni.

Il Security Gateway comunica con la rete Fabric esclusivamente tramite il client Lisp fornito dal docente, che incapsula la logica di interazione con il chaincode e la gestione delle identità crittografiche (MSP). Questo isolamento protegge il ledger da accessi diretti e non autorizzati.

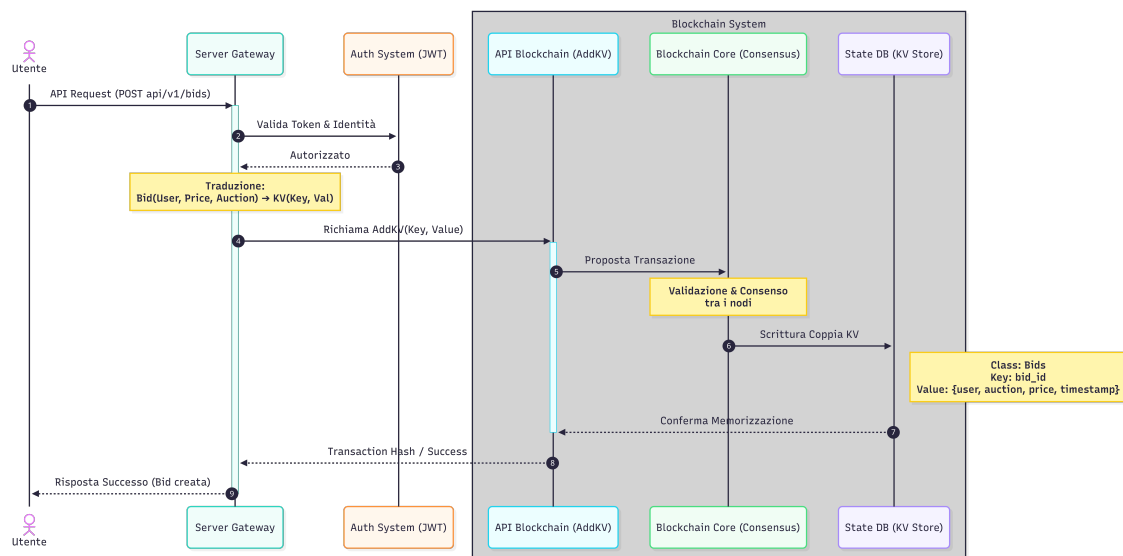


Figura 2.5: Flusso di azioni per la creazione di un'offerta

2.6 Database Off-Chain

Il database off-chain è responsabile della persistenza dei dati sensibili e delle informazioni anagrafiche degli utenti, garantendo che tali dettagli non vengano esposti direttamente sul Ledger immutabile. Il Security Gateway interagisce con questo componente per validare le credenziali di accesso e gestire i permessi, fungendo da ponte tra l'identità dell'utente sulla piattaforma web e il suo corrispettivo crittografico sulla rete Blockchain. La tecnologia adottata per l'implementazione è PostgreSQL [8].

Campo	Tipo	Note
id	string	allineato a <code>user:<uuid></code> on-chain
role	array di enum	<code>ADMIN, SELLER, BIDDER</code>
name	string	nome utente
surname	string	cognome utente
email	string	indirizzo email
birthday	date	data di nascita
cellularNumber	number	numero di cellulare
passwordHash	string	hash della password
lastLoginAt	int (epoch)	ultima data di login
createdAt	int (epoch)	data di creazione account

Tabella 2.5: Tabella Users

Per l'interazione con PostgreSQL nel contesto del framework Flask, è stata adottata la libreria **SQLAlchemy** [9], un'astrazione Object-Relational Mapping (ORM) che consente di operare su query strutturate anziché ricorrere a SQL puro. Questo approccio garantisce una maggiore sicurezza nei confronti di attacchi SQL injection e una migliore manutenibilità del codice.

Al fine di semplificare la gestione dell'infrastruttura di sviluppo e garantire la portabilità dell'applicazione, il database PostgreSQL è stato containerizzato mediante **Docker**. La configurazione del container è definita in un file `docker-compose.yml`, il quale specifica i parametri di connessione, il volume di persistenza dei dati e le variabili di ambiente necessarie. Questo approccio consente agli sviluppatori di avviare l'intera stack del database con un singolo comando, eliminando le dipendenze dall'ambiente locale e facilitando il deployment su sistemi eterogenei.

Per la messa in sicurezza delle credenziali è stato adottato l'algoritmo di hashing crittografico **bcrypt**, basato sul cifrario Blowfish. Il processo integra nativamente la generazione di un salt univoco per ogni utente, restituendo una stringa finale che concatena versione, costo, salt e digest secondo la struttura:

$$\text{pwd_hash} = \$2b\$[\text{cost}][\$[\text{salt}]]\$[\text{hash}]$$

Questo approccio garantisce protezione contro attacchi di tipo rainbow table e attacchi a forza bruta grazie al fattore di costo adattivo.

Capitolo 3

Definizione dell'infrastruttura e delle Zone di Sicurezza

L'architettura di rete del sistema **BuySellChain** è stata progettata e segmentata seguendo i principi fondamentali dello standard internazionale IEC 62443. Questa suddivisione in zone logiche di sicurezza (*Security Zones*) e condotti di comunicazione (*Conduits*) permette di isolare i sistemi critici, limitare drasticamente i movimenti laterali di un eventuale attaccante in caso di compromissione perimetrale e applicare policy di accesso granulari basate sul principio del minimo privilegio (*Least Privilege*).

3.1 Zona DMZ (Demilitarized Zone)

La DMZ funge da livello cuscinetto tra la rete pubblica (Internet) e la rete interna strettamente confinata. Il suo scopo è ricevere, filtrare e sanitizzare tutte le richieste provenienti dall'esterno prima che queste possano raggiungere i sistemi di validazione e persistenza dei dati.

Componente		Descrizione e Funzione Architetture
Reverse Proxy (NGINX)		Gestisce la terminazione TLS/SSL, isolando i server applicativi dall'esposizione diretta. Fornisce funzionalità di bilanciamento del carico e mitigazione base contro attacchi volumetrici (DDoS).
Security Gateway (Backend)		Server applicativo sviluppato in Python/Flask. Agisce come <i>Policy Enforcement Point</i> (PEP): valida i token JWT, implementa il controllo degli accessi (RBAC) e sanitizza gli input strutturali (JSON Schema) prima di invocare i comandi verso la blockchain.
Gestore delle Sessioni		Modulo integrato nel Gateway per l'emissione, la validazione e la revoca dei JSON Web Token (JWT), gestendo le transizioni di stato dell'autenticazione in modalità stateless.

Tabella 3.1: Componenti dell'infrastruttura situati nella Zona DMZ.

3.2 Zona IT / Enterprise (Restricted Zone)

Questa zona, non accessibile direttamente dall'esterno, ospita i motori di persistenza, il sistema di consenso decentralizzato e i dati sensibili. Le comunicazioni in ingresso sono consentite esclusivamente dal Security Gateway tramite protocolli e porte rigorosamente definiti.

Componente	Descrizione e Funzione Architettuale
Rete Hyperledger Fabric	Infrastruttura blockchain permissioned. Include gli <i>Orderer Nodes</i> (per l'ordinamento crittografico dei blocchi) e i <i>Peer Nodes</i> (per l'esecuzione dello Smart Contract e il mantenimento del Ledger distribuito).
Smart Contract (Chaincode)	Codice eseguibile all'interno della rete Fabric che incapsula la logica di business immutabile (es. vincoli di rilancio, finestre temporali delle aste, transizioni di stato dell'asset).
Client Lisp	Modulo software interno che agisce da intermediario tra il Security Gateway e la rete Fabric, occupandosi della firma delle transazioni tramite <i>Membership Service Provider</i> (MSP).

Tabella 3.2: Componenti dell'infrastruttura situati nella Zona IT / Enterprise.

3.3 Asset Inventory e Analisi delle Criticità

Il passo fondamentale per un'efficace modellazione delle minacce è l'identificazione e la classificazione degli asset. Di seguito vengono catalogati i componenti hardware, software e logici dell'infrastruttura BuySellChain. A ciascuno è assegnato un livello di criticità in base all'impatto di una potenziale violazione (in termini di disponibilità del servizio, integrità delle compravendite o riservatezza dei dati personali degli utenti).

Asset	Tipo	Criticità	Dipendenze e Relazioni
Smart Contract (Chaincode)	Software	Critica	Dipende direttamente dal meccanismo di consenso di Fabric. Una compromissione logica altera irreversibilmente l'esito delle aste.
Ledger Distribuito	Dati	Critica	Dipende dall'integrità dei Peer Nodes e dalle identità MSP. Contiene la <i>provenance</i> e la proprietà legale degli asset immobiliari.
Security Gateway (API)	Applicativo	Alta	Punto di esposizione pubblico verso la blockchain: valida l'autenticazione, applica RBAC e sanitizza gli input prima di invocare il Client Lisp.
Chiavi Segrete	Crittografia	Estrema	Risiedono in memoria nel Security Gateway. Se esfiltrate, permettono a un attaccante di forgiare tag e sessioni valide, compromettendo la sicurezza generale del sistema.
Frontend (Client Web)	Interfaccia	Media	Dipende dalla disponibilità del Security Gateway. Vulnerabile ad attacchi lato client (XSS, CSRF) ma non contiene logica di business fidata.
API Catasto (Sistema Terzo)	Servizio Esterno	Alta	Dipende dall'ISP e dalle CA pubbliche. Utilizzato per certificare l'esistenza degli immobili; la sua indisponibilità blocca la creazione di nuove aste.

Tabella 3.3: Inventario dettagliato degli asset del sistema BuySellChain con mappatura delle dipendenze.

3.3.1 Endpoint del Security Gateway

La tabella seguente dettaglia gli endpoint esposti dal Security Gateway, classificandoli come asset applicativi distinti in base alla loro superficie di attacco e criticità.

Endpoint	Tipo	Criticità	Note
GET/POST /admin	Applicativo	Critica	Accesso riservato ad amministratori. Esposto a privilege escalation e a tentativi di accesso non autorizzato; richiede autenticazione forte e audit logging.
GET/POST /assets	Applicativo	Media	Gestione degli asset immobiliari. Vulnerabile a injection nei campi descrittivi e a enumerazione non autorizzata degli asset altrui.
GET/POST /auctions	Applicativo	Media	Creazione e consultazione delle aste. La creazione dipende dall'API Catasto; la sua indisponibilità blocca interamente il flusso di onboarding degli immobili.
GET/POST /bids	Applicativo	Media	Invio e lettura delle offerte. Un picco anomalo di richieste POST può saturare il server o sovraccaricare la blockchain (DoS applicativo).
POST /auth	Applicativo	Alta	Emissione e validazione dei JWT. Bersaglio primario di credential stuffing, brute force e session forgery; richiede rate limiting e blocco per IP anomali.

Tabella 3.4: Endpoint del Security Gateway: classificazione per tipo e criticità.

3.4 Superficie Osservabile, Inferenza e Vettori di Attacco

La superficie osservabile rappresenta tutto ciò che un potenziale attaccante o un analista può dedurre dall'esterno dell'infrastruttura semplicemente intercettando il traffico di rete, analizzando i metadati delle comunicazioni o interagendo con le interfacce pubbliche.

Mappare questi elementi è cruciale per comprendere quali informazioni il sistema espone (anche involontariamente) e come anomalie o deviazioni comportamentali possano fungere da indicatori per formulare ipotesi di attacco. La tabella seguente unifica le osservazioni della superficie con le relative inferenze e i vettori di minaccia derivati (in linea con la metodologia STRIDE).

Asset Target	Cosa si osserva (Sintomo)	Cosa si inferisce (Ipotesi)	Possibile Attacco (Minaccia)
Endpoint /auth/login	Picco anomalo di richieste POST (es. centinaia al secondo) provenienti da IP eterogenei o singole subnet.	L'attaccante sta testando liste di credenziali preesfiltrate o tentando di forzare una password.	Credential Stuffing o Brute Force (Spoofing).
Endpoint /admin/users	Picco anomalo di richieste GET provenienti da IP eterogenei o singole subnet.	Tentativo di enumerare gli utenti del sistema.	Enumeration.
Endpoint /bids, /auctions, /assets	Picco di richieste POST agli endpoint di creazione dati.	Tentativo di saturare il server oppure sovraccaricare la blockchain.	Denial of Service (DoS) applicativo o Overflood.
Security Gateway (API)	Caratteri di escape insoliti, cicli annidati o tentativi di iniezione nei payload JSON in ingresso.	L'attaccante cerca di bypassare il parser per eseguire comandi diretti sul Client Lisp.	Lisp Command Injection.
Sessioni Utente (JWT)	Modifica del payload Base64 del JWT (es. alterazione del campo <i>role</i> da <i>Bidder</i> ad <i>Admin</i>).	Un utente malintenzionato sta testando la robustezza dell'algoritmo di firma HMAC.	Privilege Escalation o Session Forgery.
API Catasto (connessioni in uscita)	Traffico anomalo dal Security Gateway verso IP sconosciuti al di fuori dei domini governativi previsti.	Il Gateway è stato compromesso e sta esfiltrando dati, oppure è vittima di un attacco SSRF.	Server-Side Request Forgery o Data Exfiltration (Information Disclosure).
Frontend (Interfaccia)	Inserimento di tag HTML o script JavaScript nei campi descrittivi degli immobili in vendita.	Ricerca di vulnerabilità nella sanitizzazione lato server dei template Jinja2.	Cross-Site Scripting (XSS) e furto di token.

Tabella 3.5: Threat Model unificato: dalla superficie osservabile all'inferenza dell'attacco.

3.5 Grafo dell'Infrastruttura e Flussi di Comunicazione

Per agevolare l'analisi dei rischi e l'implementazione dei controlli di sicurezza, viene presentata la topologia dei flussi di rete. Questa scomposizione evidenzia i nodi di origine e destinazione, i confini di trust attraversati, i protocolli crittografici necessari e l'intensità fisiologica attesa per ciascun condotto (utile a stabilire una *baseline* comportamentale per i sistemi di Intrusion Detection).

Ogni deviazione da questi flussi autorizzati (ad esempio, il Reverse Proxy NGINX che tenta di comunicare direttamente con la blockchain) deve essere considerata un'anomalia severa.

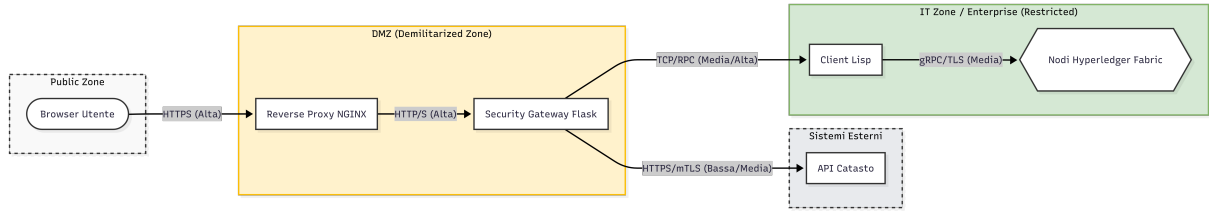


Figura 3.1: Grafo dell'infrastruttura BuySellChain con zone di sicurezza e flussi autorizzati.

Sorgente	Destinazione	Protocollo	Frequenza	Motivazione / Logica di Business
Browser Utente	Reverse Proxy (DMZ)	HTTPS	Alta	Navigazione piattaforma, consultazione grid aste, login e invio offerte. Traffico cifrato TLS 1.3.
Reverse Proxy	Security Gateway	HTTP/S	Alta	Inoltro delle richieste decifrate (o ricifrate internamente) verso il server Flask per l'elaborazione.
Security Gateway	Client Lisp (IT Zone)	TCP	Media/Alta	Traduzione delle chiamate REST in primitive (AddKV , GetKV) per l'interazione con la blockchain.
Client Lisp	Nodi Fabric (IT Zone)	gRPC/TLS	Media	Invocazione dello Smart Contract, proposta di transazioni e interrogazione dello stato del Ledger.
Security Gateway	API Catasto (Esterno)	HTTPS/mTLS	Bassa/Media	Chiamata sincrona per la certificazione e la validazione dei dati di un immobile in fase di creazione asta.

Tabella 3.6: Mappatura dettagliata dei flussi di comunicazione autorizzati (Conduits) tra gli asset.

3.6 Dall'Architettura alla Baseline Comportamentale (Operating Envelope)

L'obiettivo di questa fase è usare la topologia di BuySellChain per definire cosa costituisce un comportamento di rete *normale*. La normalità non è un punto singolo, ma un inviluppo operativo continuo (*operating envelope*). La classificazione degli eventi deve quindi distinguere tra **Normal**, **Deviazione Lieve**, **Deviazione Forte** e **Unknown**.

3.6.1 Definizione della Baseline Qualitativa

Le comunicazioni attese vengono modellate come relazioni lecite tra asset, con frequenza temporale e intensità prevista.

Relazione Normale	Quando	Intensità	Note
Frontend Web → Security Gateway	Sempre	Alta	Navigazione utenti, login, invio offerte.
Security Gateway → Rete Fabric (via Client Lisp)	Sempre	Media	Invocazione Smart Contract per registrazione aste e offerte.
Security Gateway → API Catasto	Giorno/Notte	Bassa/Media	Verifica validità immobili, chiamate asincrone e sincrone.
Client esterno → Server Gateway	Mai	Nulla	Deviazione topologica grave. Client non-browser che tentano di invocare direttamente lo Smart Contract.
<i>Flussi utente (Seller/Buyer) — baseline applicativa</i>			
Login/Registrazione → Crea Asset → Crea Asta	Sì	Normale	Flusso canonico del venditore: autenticazione, onboarding immobile, apertura asta.
Login/Registrazione → Crea Asta	Sì	Normale	Flusso abbreviato se l'asset è già presente nel ledger.
Home Page (no login) → Crea Asta / Crea Asset	No	Deviazione	Accesso a risorse protette senza autenticazione. Contromisura: redirect al login o risposta 403 Forbidden .

Tabella 3.7: Baseline qualitativa dei flussi leciti e applicativi (Operating Envelope).

Capitolo 4

Logger

Per completare il modello di minaccia, è necessario definire un sistema di logging centralizzato che raccolga i dati di telemetria da tutti i componenti critici (Security Gateway, Client Lisp, Nodi Fabric) e li normalizzi in un formato standardizzato (es. JSON con timestamp ISO8601, ID transazione, livello di severità). Questi log devono essere arricchiti con metadati contestuali (es. geolocalizzazione IP, user agent, payload size) per facilitare l'analisi comportamentale e la correlazione degli eventi. Il sistema di logging deve essere progettato per resistere a tentativi di manomissione (es. log immutabili, firma digitale dei log) e per supportare query ad-hoc per l'investigazione post-incident.

Come vedremo, il sistema di logging scrive i record in blockchain, garantendo così l'immutabilità e la tracciabilità completa degli eventi di sicurezza. Questo approccio innovativo consente di superare le limitazioni dei tradizionali sistemi di logging centralizzati, offrendo una soluzione altamente resistente alla manomissione e alla perdita dei dati, e facilitando l'analisi forense in caso di incidenti di sicurezza.

Prima di definire il sistema di logging, abbiamo effettuato una migrazione della tabella **Users**, dal database off-chain Postgres, alla blockchain Hyperledger Fabric. Questa migrazione è stata necessaria per garantire la coerenza e l'integrità dei dati degli utenti, sfruttando le caratteristiche di immutabilità e decentralizzazione della blockchain. La tabella **Users** ora risiede all'interno della blockchain, consentendo una gestione più sicura e trasparente delle informazioni degli utenti, e facilitando l'implementazione di meccanismi di autenticazione e autorizzazione basati su identità digitali registrate sulla blockchain.

4.1 Definizione Sistema di Logging

Innanzitutto è stato necessario definire quali metadati devono essere raccolti nei log per garantire un'analisi efficace degli eventi di sicurezza. I log devono includere informazioni quali timestamp, livello di criticità, geolocalizzazione IP, user agent, metodo HTTP e payload size. Questi metadati consentono di contestualizzare gli eventi e facilitano l'identificazione di pattern sospetti o anomalie nel comportamento degli utenti o dei sistemi.

Memorizzare in **chiaro** l'IP e lo user agent di chi interagisce con il sistema è controproducente per la privacy degli utenti, ma è necessario per l'analisi comportamentale e la correlazione degli eventi. Per mitigare questo rischio, i log vengono cifrati prima di essere

scritti sulla blockchain, utilizzando una chiave simmetrica gestita in modo sicuro all'interno del Security Gateway. In questo modo, anche se i log sono immutabili e trasparenti sulla blockchain, le informazioni sensibili rimangono protette e accessibili solo agli operatori autorizzati che dispongono della chiave di decrittazione.

Di seguito, è mostrata la struttura dei log:

```
1 {  
2  
3   "id": hash(description, created_at, level, from_ip, user_agent, method, payload_size)  
4   ,  
5   "description": MessageType,  
6   "created_at": timestamp,  
7   "level": INFO/ALERT,  
8   "from_ip": AES-256-GCM(source IPv4),  
9   "method": HTTP method,  
10  "user_agent": AES-256-GCM(useragent),  
11  "payload_size": N Kb size of payload  
}
```

In particolare, il logger tiene traccia delle seguenti azioni:

- **Login/Registrazione:** ogni tentativo di autenticazione, riuscito o fallito, viene registrato con il relativo user agent e IP di origine. Inoltre, viene segnalato ogni nuovo utente registrato nel sistema.
- **Creazione Asta:** quando un utente crea una nuova asta, viene loggato l'ID dell'asta, l'ID dell'utente e i dettagli dell'immobile.
- **Creazione Asset:** quando un utente crea una nuovo asset, viene loggato l'ID dell'asset, l'ID dell'utente e i dettagli dell'immobile.
- **Offerte:** ogni offerta fatta su un'asta viene registrata con l'ID dell'offerta, l'ID dell'utente, l'importo e l'ID dell'asta.
- **Errori e Anomalie:** qualsiasi errore del sistema o comportamento anomalo (es. tentativi di accesso non autorizzati) viene loggato con un livello di criticità adeguato (INFO, ALERT).

Come accennato sopra, i campi `from_ip` e `user_agent` vengono cifrati prima di essere memorizzati sulla blockchain, garantendo così la protezione della privacy degli utenti pur mantenendo la capacità di analizzare i log per identificare comportamenti sospetti o pattern di attacco. Solo l'admin, il quale ha accesso alle api di lettura dei log è in grado di leggere in chiaro questi campi, e solo in caso di necessità di investigazione post-incident.

4.2 Esempi di Log e Analisi Comportamentale

Di seguito, vediamo alcuni esempi:

Log Cifrato in Blockchain Se eseguita una `GetKV` per leggere un log relativo alla registrazione di un nuovo utente, il record restituito sarà simile al seguente (con i campi sensibili cifrati):

```
1 {
2   "success": true,
3   "message": "Ok",
4   "answer": {
5     "value": {
6       "id": "6a8e249114664e905f84d83d7c992a860e950f5baf04894679581a05bd69d9e1",
7       "description": "Nuovo utente registrato: user123",
8       "created_at": "2026-05-15T11:47:53.512788",
9       "level": "INFO",
10      "from_ip": "1IBpH+5m+0HAsrUVwVV1l9i0Yvu2KSqVGk8zxymgJpVQq0FKP70dRgg=",
11      "user_agent": "2j0pqAPMUAad5KVGvUzR8gc9ZhmmWz3uYKvFNLeMaCe0mSQeuoY9ykAcUAiEnUC",
12      "method": "POST",
13      "payload_size": 45
14    },
15    "key": [
16      "6a8e249114664e905f84d83d7c992a860e950f5baf04894679581a05bd69d9e1"
17    ],
18    "class": "Logs"
19  }
20 }
```

Log Decifrato per Analisi Se l'admin esegue una query per leggere i log, i campi cifrati vengono decifrati e restituiti in chiaro, consentendo di analizzare le informazioni per identificare eventuali anomalie o pattern sospetti:

```
1 {
2   "created_at": "2026-05-15T11:47:53.512788",
3   "from_ip": "127.0.0.1",
4   "id": "6a8e249114664e905f84d83d7c992a860e950f5baf04894679581a05bd69d9e1",
5   "level": "INFO",
6   "message": "Nuovo utente registrato: user123",
7   "method": "POST",
8   "payload_size": 45,
9   "user_agent": "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
10 }
```

4.2.1 User Agent Metadata Role

Un metadato fondamentale per l'analisi comportamentale è lo `user_agent`. Questo ci dice da quale software/client vengono fatte le richieste al server. Se vediamo un user agent insolito (es. un tool di pen testing o un bot), questo può essere un indicatore di attività malevola. Ad esempio, se notiamo un picco di richieste con user agent "sqlmap" o "Nmap", potremmo inferire che qualcuno sta cercando di identificare vulnerabilità nel sistema. Allo stesso modo, se vediamo user agent che non corrispondono a browser comuni (es. "curl/7.68.0"), questo potrebbe indicare tentativi di accesso automatizzati o scriptati, che potrebbero essere utilizzati per attacchi di forza bruta o scraping dei dati.

```
1 {  
2   "created_at": "2026-05-15T12:41:24.896036",  
3   "from_ip": "127.0.0.1",  
4   "id": "9f1b99fc04ee6a489da4901ab9eb226cfdbf930877f7422752c21a609a7d581a",  
5   "level": "ALERT",  
6   "message": "Accesso non autorizzato /auctions/create",  
7   "method": "GET",  
8   "payload_size": 0,  
9   "user_agent": "curl/8.15.0"  
10 }
```

Come possiamo vedere nell'esempio sopra, il log indica un tentativo di accesso non autorizzato all'endpoint `/auctions/create` utilizzando l'user agent "curl/8.15.0". Questo potrebbe essere un indicatore di un attacco automatizzato e dovrebbe essere investigato ulteriormente per determinare se si tratta di un'attività malevola o di un errore legittimo.

Inoltre, questo log ci dice anche che all' endpoint `/auctions/create` è stato fatto l'accesso senza autenticazione, il che è una deviazione dalla baseline operativa definita in precedenza, dove l'accesso a questo endpoint dovrebbe essere consentito solo agli utenti autenticati. Questo tipo di deviazione è un chiaro indicatore di un potenziale attacco e dovrebbe essere trattato con la massima attenzione.

4.3 Monitoraggio Autenticazione & Admin

Il sistema di logging tiene traccia di tutti i tentativi di autenticazione, sia riusciti che falliti. Questo è fondamentale per identificare potenziali attacchi di forza bruta o credential stuffing. Ad esempio, se vediamo un picco di tentativi di login falliti da un singolo IP o da una serie di IP correlati, questo potrebbe indicare un attacco in corso.

Inoltre, il sistema di logging consente di monitorare le tendenze nel tempo, identificando periodi di attività sospetta o anomalie nei pattern di autenticazione. Ad esempio, se vediamo un aumento improvviso dei tentativi di login durante un periodo in cui normalmente ci sarebbe poca attività, questo potrebbe essere un indicatore di un attacco in corso.

Per non dare troppe informazioni nel caso in cui la blockchain venga compromessa, i log di autenticazione falliti conterranno solo un messaggio generico che indica un tentativo di login fallito. In questo modo, anche se un attaccante riesce ad accedere ai log, non avrà informazioni dettagliate sui tentativi di autenticazione falliti, riducendo il rischio di ulteriori attacchi basati su queste informazioni.

Infine, il sistema di logging consente di monitorare le attività degli amministratori, registrando ogni azione eseguita dagli utenti con privilegi elevati. Questo è fondamentale per identificare potenziali abusi di privilegi o attività sospette da parte degli amministratori

stessi. Ad esempio, se vediamo un amministratore che accede a dati sensibili o esegue azioni non autorizzate, questo potrebbe essere un indicatore di un abuso di privilegi o di un attacco interno.

Bibliografia

- [1] The Hyperledger Foundation, *Hyperledger Fabric Documentation*. The Linux Foundation, 2024. Versione 2.5 LTS. Accesso: Gennaio 2026.
- [2] A. Ronacher, “Jinja2: A full-featured template engine for Python.” <https://jinja.palletsprojects.com/>, 2026. Accessed: 2026-02-12.
- [3] C. Porzio, “Alpine.js: A rugged, minimal framework for composing JavaScript behavior in your markup.” <https://alpinejs.dev/>, 2026. Accessed: 2026-02-12.
- [4] Python Software Foundation, “Python language reference, version 3.12,” 2023. Disponibile su python.org.
- [5] Pallets Projects, “Flask: A microframework for python based on werkzeug, jinja 2 and good intentions,” 2010. Versione 3.0.x.
- [6] L. Turner, “Flask-jwt-extended: Open source extension for flask adding support for json web tokens.” <https://github.com/vimalloc/flask-jwt-extended>, 2023.
- [7] M. B. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT).” RFC 7519, May 2015.
- [8] PostgreSQL Global Development Group, *PostgreSQL: The World’s Most Advanced Open Source Relational Database*. PostgreSQL Global Development Group, 2026. Accessed: 2026-02-09.
- [9] M. Bayer, “SQLAlchemy.” <https://www.sqlalchemy.org/>, 2026. Accessed: 2026-02-12.